

Design and Development of a 5-DOF Pick and Place Manipulator.

October 2, 2025

1 Background

Robotic manipulators are widely used in automation for handling, assembly, and packaging. A 5-DOF arm offers full spatial control, allowing objects to be positioned and oriented flexibly. While industrial robots are effective, they are often costly and complex for small-scale use. Advances in microcontrollers, low-cost actuators, and 3D printing make it possible to build simpler, affordable manipulators. This project applies mechatronics principles to design a functional 6-DOF pick-and-place system as a practical educational prototype.

2 Aim

To design and develop a cost-effective 5-DOF robotic arm with a gripper that demonstrates accurate pick-and-place operations using affordable components.

3 Objectives

- To design a low-cost 5-DOF robotic arm with modular structure for easy adaptation.
- To implement real-time multi-axis control using an STM32 for precise motion.
- To integrate vision for object detection and demonstrate accurate pick-and-place.

4 Scope and Limitations

The project focuses on designing, fabricating, and testing a low-cost 5-DOF manipulator capable of performing basic pick-and-place tasks. The prototype is restricted to light payloads (< 500 g) and operates in a controlled environment with limited accuracy compared to industrial robots.

5 System Design Overview

The system consists of a custom-designed 5-DOF robotic arm with a gripper end effector, integrated with an embedded control system based on an STM32 microcontroller (Cortex-M0 core) for precise multi-axis motion. A camera module connected to a host PC provides machine vision capabilities, enabling object detection and localization. The vision system supplies the microcontroller with the coordinates of target objects, which are then used for pick-and-place operations. At the architectural level, the system can be described as a closed-loop mechatronic platform where sensing (camera), computation (PC and STM32), actuation (servo-driven joints), and end-effector manipulation (gripper) interact seamlessly. Functionally, the manipulator executes motion commands derived from G-code instructions while continuously receiving updated position data from the vision system. A block diagram representation of the architecture highlights these subsystems and the flow of information between them.

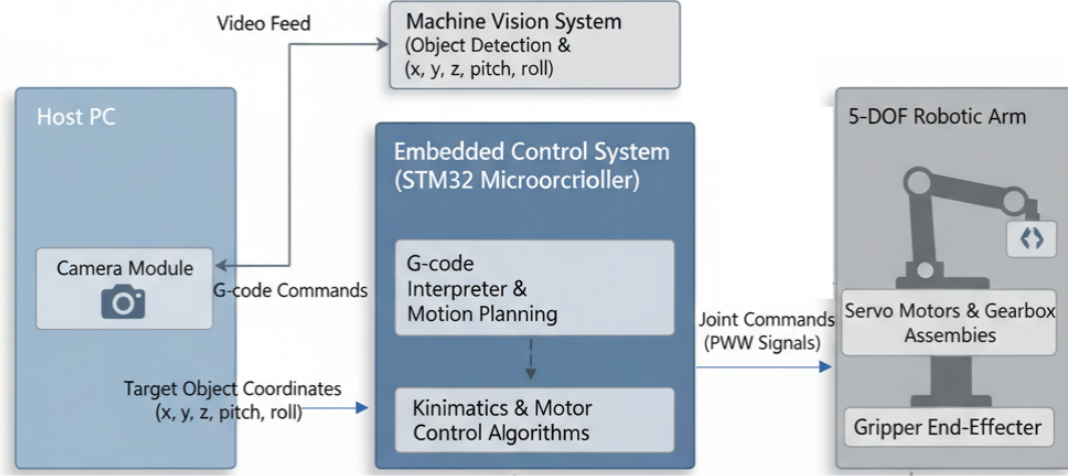


Figure 1: A block diagram of the Beta-Arm system

6 Mechanics

The robotic manipulator is a 5-DOF articulated arm composed of five revolute joints and corresponding links (a_1 – a_5) that form its kinematic chain. Each joint provides a single rotational degree of freedom, enabling coordinated multi-axis motion suitable for basic pick-and-place tasks within a controlled workspace. A gripper end effector is mounted at the terminal link to handle lightweight objects, and the modular structure of the arm allows for easy mechanical adaptation and future extensions.

6.1 Kinematics

6.1.1 Forward Kinematics

The kinematic model of the manipulator was developed using the defined link lengths (a_1 – a_5) and joint constraints. The structure was modeled in Autodesk Inventor to define geometry and assembly, while MATLAB was used to analyze workspace reach and validate joint limits. The resulting kinematic framework forms the foundation for subsequent motion planning and control.

The forward kinematics of the robotic manipulator involves systematically describing the relationship between consecutive links and joints in order to determine the position and orientation of the end-effector relative to the base frame. This process makes use of link parameters, joint definitions, rotational matrices, displacement vectors, and the Denavit–Hartenberg (D–H) convention, culminating in the construction of homogeneous transformation matrices.

1. **Link Parameters:** Define the physical dimensions of the manipulator, such as link lengths a_1 – a_5 , which form the kinematic chain:

$$a_1 = 150.933 \text{ mm}, \quad a_2 = 128.914 \text{ mm}, \quad a_3 = 167.164 \text{ mm}, \quad a_4 = 50.056 \text{ mm}, \quad a_5 = 75.637 \text{ mm}$$

2. **Joint Limits:** The manipulator consists of five revolute joints, each providing one degree of freedom (DOF). Each joint is constrained within the range:

$$T_0, T_1, T_2, T_3, T_4 \in [-90^\circ, 90^\circ]$$

3. **Rotational Matrices:** Orientation between consecutive link frames is defined as:

$$R_1^0 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} R_y, \quad R_2^1 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} R_z,$$

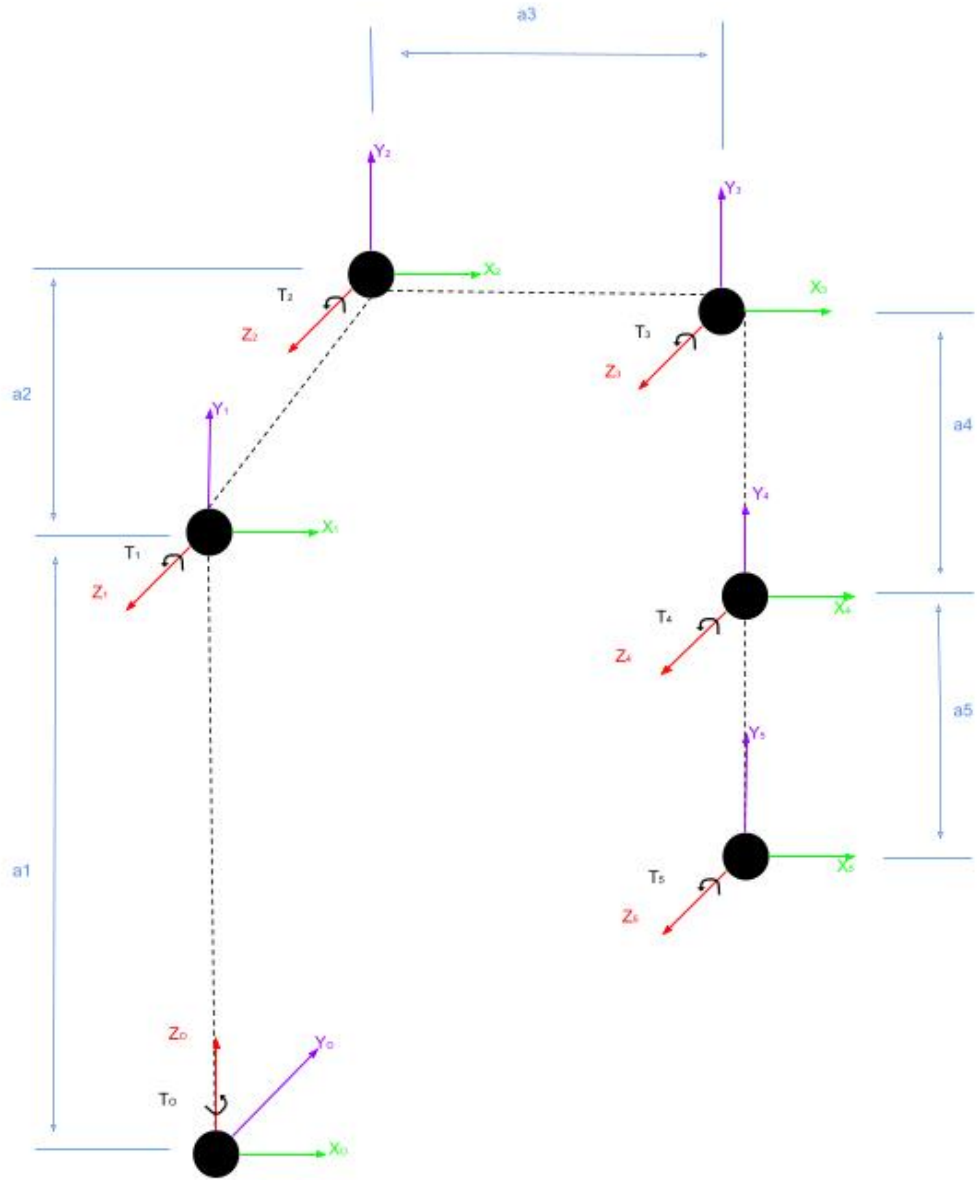


Figure 2: A kinematic diagram of the 5-DOF manipulator.

$$\begin{aligned}
R_3^2 &= \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} R_z, & R_4^3 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} R_y \\
R_5^4 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} R_z, & R_5^0 &= \begin{bmatrix} 0 & 0 & 1 \\ 0 & -1 & 0 \\ 1 & 0 & 0 \end{bmatrix} R_x
\end{aligned}$$

4. **Displacement Vectors:** Capture the relative translation between link frames, based on link offsets:

$$d_1^0 = \begin{bmatrix} 0 \\ 0 \\ a_1 \end{bmatrix}, \quad d_2^1 = \begin{bmatrix} a_2 \\ 0 \\ 0 \end{bmatrix}, \quad d_3^2 = \begin{bmatrix} a_3 \\ 0 \\ 0 \end{bmatrix}, \quad d_4^3 = \begin{bmatrix} a_4 \\ 0 \\ 0 \end{bmatrix}, \quad d_5^4 = \begin{bmatrix} 0 \\ 0 \\ a_5 \end{bmatrix}$$

5. **Denavit–Hartenberg Parameters in Tabular format:** Summarizes all kinematic parameters $(\theta_i, \alpha_i, a_i, d_i)$ in a standardized form for the 5-DOF manipulator (see Table 1).

i	θ_i	α_i	a_i	d_i
1	θ_1	90°	0	a_1
2	θ_2	0	a_2	0
3	θ_3	0	a_3	0
4	θ_4	90°	0	0
5	θ_5	0	0	$a_4 + a_5$

Table 1: Denavit–Hartenberg parameters for the robotic arm joints.

6. **Homogeneous Transformation Matrices:** Combine the rotational and translational effects into a single 4×4 matrix, enabling the computation of the end-effector pose by successive multiplications:

$$c_i \equiv \cos \theta_i, \quad s_i \equiv \sin \theta_i \quad (i = 1, \dots, 5).$$

$${}^0T_1 = \begin{bmatrix} c_1 & -s_1 \cos 90^\circ & s_1 \sin 90^\circ & 0 \cdot c_1 \\ s_1 & c_1 \cos 90^\circ & -c_1 \sin 90^\circ & 0 \cdot s_1 \\ 0 & \sin 90^\circ & \cos 90^\circ & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

where $d_1 = a_1$ (per table).

$$\begin{aligned}
{}^1T_2 &= \begin{bmatrix} c_2 & -s_2 \cos 0 & s_2 \sin 0 & a_2 c_2 \\ s_2 & c_2 \cos 0 & -c_2 \sin 0 & a_2 s_2 \\ 0 & \sin 0 & \cos 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} c_2 & -s_2 & 0 & a_2 c_2 \\ s_2 & c_2 & 0 & a_2 s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \\
{}^2T_3 &= \begin{bmatrix} c_3 & -s_3 \cos 0 & s_3 \sin 0 & a_3 c_3 \\ s_3 & c_3 \cos 0 & -c_3 \sin 0 & a_3 s_3 \\ 0 & \sin 0 & \cos 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} c_3 & -s_3 & 0 & a_3 c_3 \\ s_3 & c_3 & 0 & a_3 s_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \\
{}^3T_4 &= \begin{bmatrix} c_4 & -s_4 \cos 90^\circ & s_4 \sin 90^\circ & 0 \cdot c_4 \\ s_4 & c_4 \cos 90^\circ & -c_4 \sin 90^\circ & 0 \cdot s_4 \\ 0 & \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} c_4 & 0 & s_4 & 0 \\ s_4 & 0 & -c_4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
\end{aligned}$$

$${}^4T_5 = \begin{bmatrix} c_5 & -s_5 \cos 0 & s_5 \sin 0 & 0 \cdot c_5 \\ s_5 & c_5 \cos 0 & -c_5 \sin 0 & 0 \cdot s_5 \\ 0 & \sin 0 & \cos 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} c_5 & -s_5 & 0 & 0 \\ s_5 & c_5 & 0 & 0 \\ 0 & 0 & 1 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

where $d_5 = a_4 + a_5$ (per table).

$$\boxed{{}^0T_5 = {}^0T_1 {}^1T_2 {}^2T_3 {}^3T_4 {}^4T_5}$$

$${}^0T_5 = \begin{bmatrix} R_0^5 & p_0^5 \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \text{with} \quad R_0^5 = R_1 R_2 R_3 R_4 R_5, \quad p_0^5 = (\text{resulting top-right } 3 \times 1 \text{ block after multiplication}).$$

6.1.2 Inverse Kinematics

The inverse kinematics (IK) solution for the given 5-DOF serial manipulator is derived analytically using the Denavit-Hartenberg (DH) parameters. Assume the end-effector pose is specified by the homogeneous transformation matrix ${}^0T_5 = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix}$, where $\mathbf{p} = [p_x, p_y, p_z]^\top$ is the position and $\mathbf{R} = [r_{ij}]$ ($i, j = 1, 2, 3$) is the rotation matrix. The solution proceeds in two stages: (1) solving for the first three joint angles $\theta_1, \theta_2, \theta_3$ to position the wrist center, and (2) solving for the last two angles θ_4, θ_5 to achieve the desired orientation. Two solutions may exist due to the elbow singularity (elbow-up/elbow-down configurations).

1. **Computing the Wrist Center Position.** The wrist center position $\mathbf{p}_w = [p_{wx}, p_{wy}, p_{wz}]^\top$ is obtained by back-projecting from the end-effector along the approach vector (third column of $\mathbf{R}$):

$$p_{wx} = p_x - d_5 r_{13}, \quad p_{wy} = p_y - d_5 r_{23}, \quad p_{wz} = p_z - d_5 r_{33}, \quad (1)$$

where $d_5 = a_4 + a_5$.

2. **Solving for θ_1 (Base Rotation).** The first joint angle is determined from the projection in the xy -plane:

$$\theta_1 = \text{atan2}(p_{wy}, p_{wx}), \quad (2)$$

where $\text{atan2}(y, x)$ is the two-argument arctangent function.

$$\text{Let } c_1 = \cos \theta_1 = \frac{p_{wx}}{\sqrt{p_{wx}^2 + p_{wy}^2}} \text{ and } s_1 = \sin \theta_1 = \frac{p_{wy}}{\sqrt{p_{wx}^2 + p_{wy}^2}}.$$

The radial distance in the xy -plane is $x_c = \sqrt{p_{wx}^2 + p_{wy}^2}$.

3. **Solving for θ_3 (Elbow Angle).** Using the law of cosines in the shoulder-elbow-wrist triangle (with vertical offset $z' = p_{wz} - a_1$ and planar distance $s = \sqrt{x_c^2 + (z')^2}$):

$$\cos \theta_3 = \frac{x_c^2 + (p_{wz} - a_1)^2 - a_2^2 - a_3^2}{2a_2a_3}. \quad (3)$$

Then,

$$\sin \theta_3 = \pm \sqrt{1 - \cos^2 \theta_3}, \quad (4)$$

yielding two possible solutions (positive for elbow-down, negative for elbow-up).

4. **Solving for θ_2 (Shoulder Angle).** With θ_3 known, define $A = a_2 + a_3 \cos \theta_3$ and $B = a_3 \sin \theta_3$. Then,

$$\theta_2 = \text{atan2}(z', x_c) - \text{atan2}(B, A), \quad (5)$$

where $z' = p_{wz} - a_1$.

Inverse Kinematics - Geometric Solution

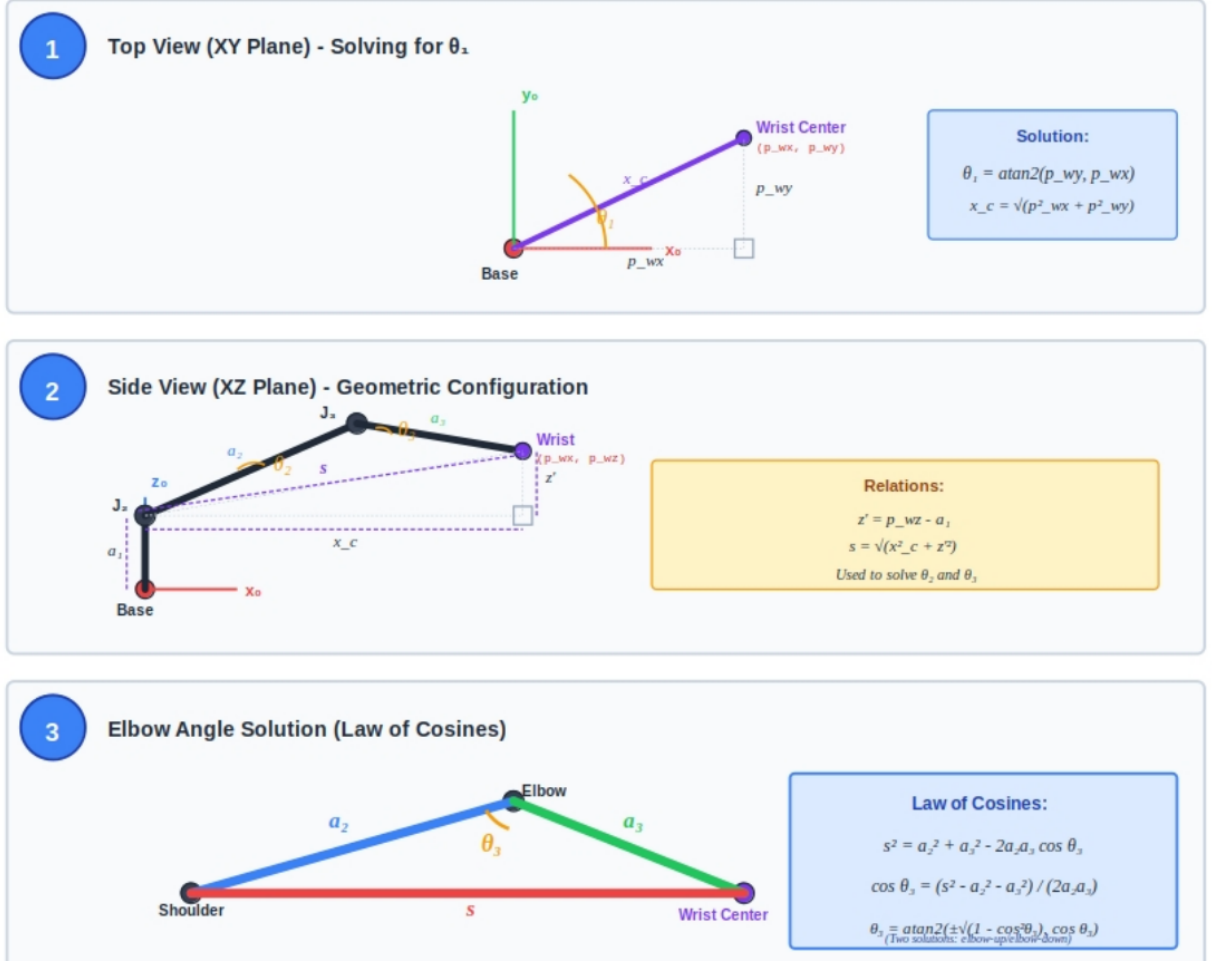


Figure 3: Inverse kinematic diagrams of the 5-DOF manipulator.

5. **Orientation Consistency Check.** Computing the columns of ${}^0\mathbf{R}_3$ (rotation matrix to frame 3) using the solved $\theta_1, \theta_2, \theta_3$:

$$\mathbf{u} = \begin{bmatrix} -\sin \theta_2 \sin \theta_3 \cos \theta_1 + \cos \theta_1 \cos \theta_2 \cos \theta_3 \\ -\sin \theta_1 \sin \theta_2 \sin \theta_3 + \sin \theta_1 \cos \theta_2 \cos \theta_3 \\ \sin \theta_2 \cos \theta_3 + \sin \theta_3 \cos \theta_2 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} -\sin \theta_2 \cos \theta_1 \cos \theta_3 - \sin \theta_3 \cos \theta_1 \cos \theta_2 \\ -\sin \theta_1 \sin \theta_2 \cos \theta_3 - \sin \theta_1 \sin \theta_3 \cos \theta_2 \\ -\sin \theta_2 \sin \theta_3 + \cos \theta_2 \cos \theta_3 \end{bmatrix}. \quad (6)$$

The second column of $\mathbf{R}$ must satisfy $\mathbf{r}_{:2} = [s_1, -c_1, 0]^\top$ and $r_{32} = 0$ for a valid solution; otherwise, no real solution exists.

6. **Solving for $\phi = \theta_4 + \theta_5$ (Wrist Aggregate Angle).** Using the third row of $\mathbf{R}$:

$$\phi = (\theta_2 + \theta_3) - \text{atan2}(r_{31}, r_{33}). \quad (7)$$

7. **Solving for θ_4 (Wrist Pitch).** The approach vector $\mathbf{n} = [r_{13}, r_{23}, r_{33}]^\top$ satisfies $\mathbf{n} = \sin \theta_4 \mathbf{u} - \cos \theta_4 \mathbf{v}$. Thus,

$$\sin \theta_4 = \mathbf{n}^\top \mathbf{u}, \quad \cos \theta_4 = -\mathbf{n}^\top \mathbf{v}, \quad (8)$$

and

$$\theta_4 = \text{atan2}(\mathbf{n}^\top \mathbf{u}, -\mathbf{n}^\top \mathbf{v}). \quad (9)$$

8. **Solving for θ_5 (Wrist Roll).** Finally,

$$\theta_5 = \phi - \theta_4. \quad (10)$$

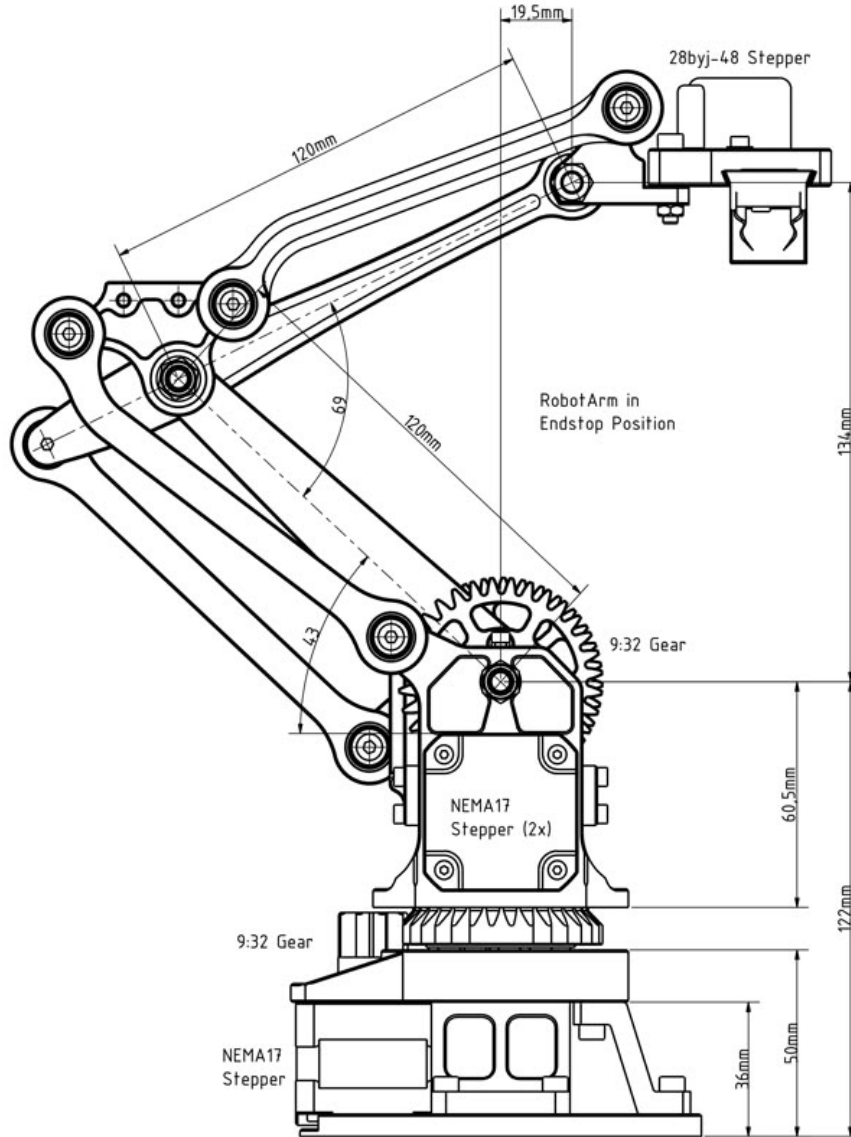


Figure 4: 2-D CAD drawing

6.2 Mechanical-CAD

The mechanical design phase of the Beta-Arm project harnessed Autodesk Inventor as the primary MCAD platform to conceptualize and refine a lightweight, 5-DOF serial robotic arm optimized for 3D printing and servo integration, transforming initial kinematic sketches into a fully assemblable prototype with minimal material waste and enhanced structural performance. The design workflow commenced in Inventor's part modeling environment, where over 25 individual components were meticulously crafted, including the robust base mount for stable platform attachment, the ergonomic shoulder bracket to distribute torque loads, segmented upper arm links for flexibility, intricate forearm elements to achieve the desired reach, a modular wrist assembly for precise orientation control, and duplicated 29-tooth spur gears to implement backlash-minimizing reduction ratios. Upon finalizing the virtual model, the export phase tailored outputs for downstream manufacturing, generating high-resolution .stl meshes.

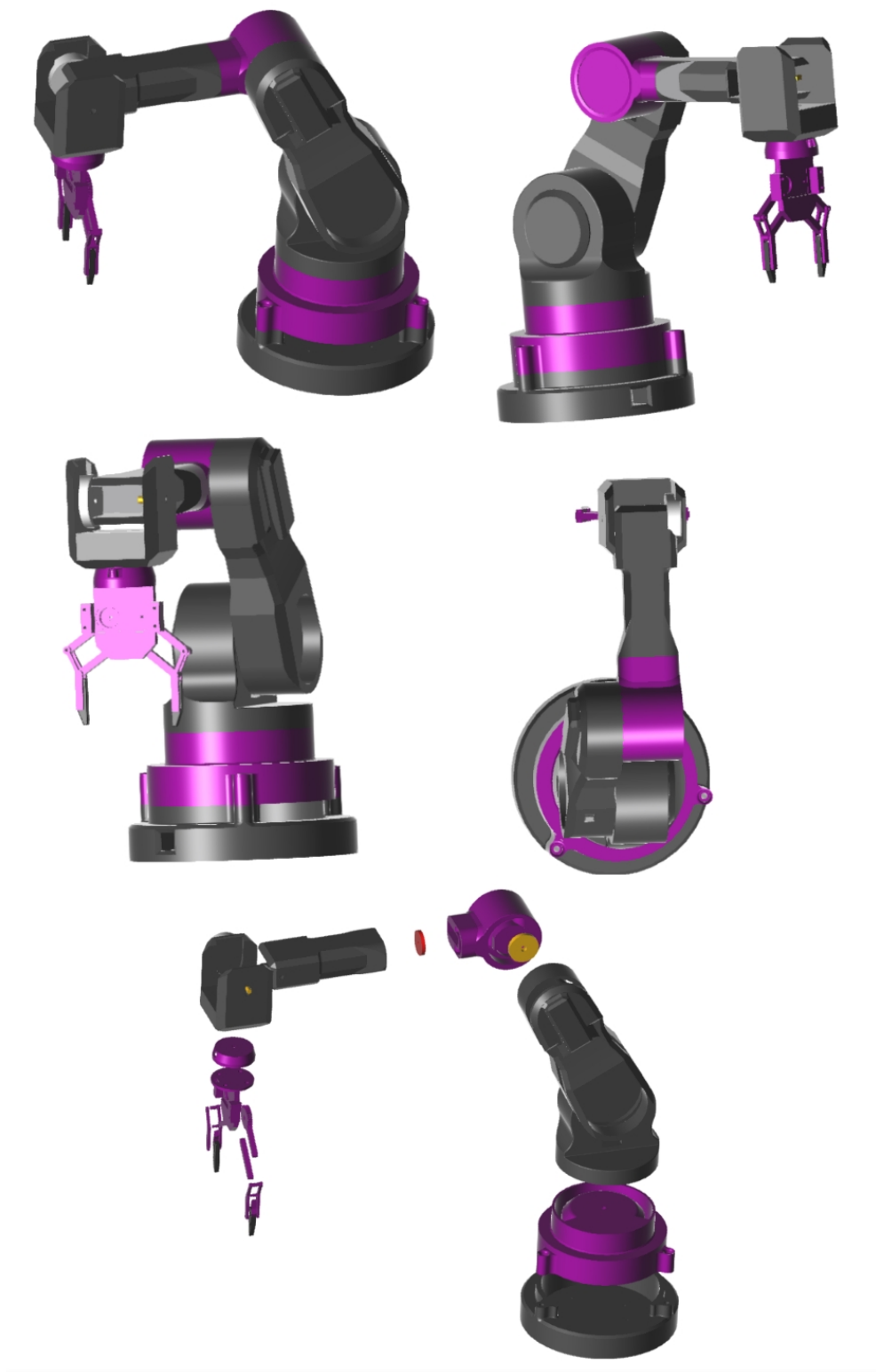


Figure 5: 3-D projections of the 5-DOF manipulator.

Servo Motor Driver Circuit

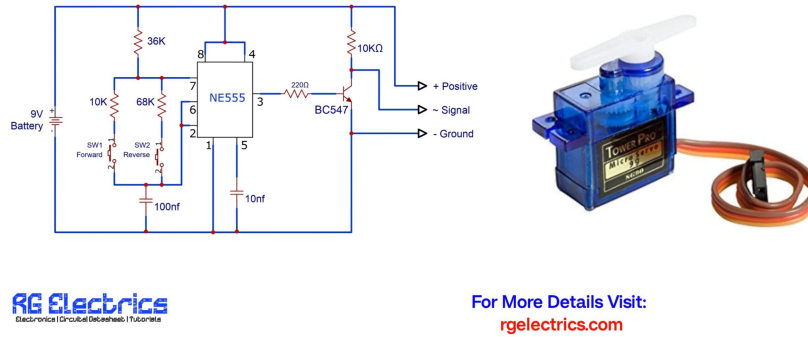


Figure 6: Circuit schematic

Servo Motor Driver Circuit

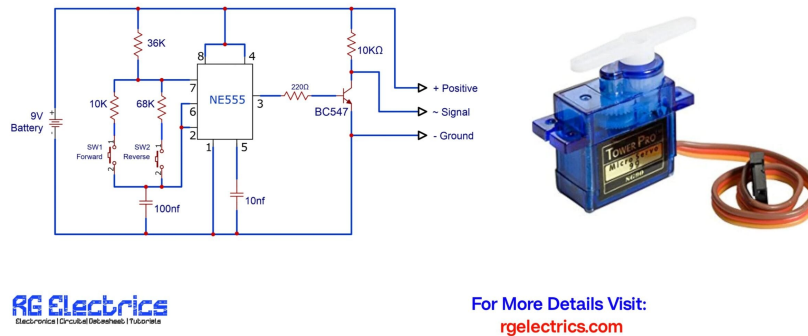


Figure 7: Circuit schematic

7 Electronics

The electronics subsystem of the Beta-Arm integrates low-cost, readily available components to enable precise servo actuation, embedded control, and computer vision integration. Key selections prioritize compatibility with the STM32 microcontroller, power efficiency, and scalability for future enhancements like wireless communication.

7.1 Circuit Design and Wiring

The circuit employs a modular breadboard layout to minimize soldering and facilitate prototyping. The STM32F030C6 interfaces with the PCA9685 via I²C (SDA/SCL pins) for PWM servo control, with five MG996R servos and one MG90 connected to channels 0–5, sharing a common 5 V rail from the power adapter. UART (TX/RX) on the STM32 links to the host PC for command reception from Python scripts, while the ESP32-CAM connects via USB or serial for image streaming to the host for YOLO processing. Power distribution uses the barrel jack to the breadboard rails.

7.2 Bill of Materials (BOM)

The BOM lists essential components, estimated costs, and quantities, totaling under \$50 for core electronics.

Component	Qty	Specification
STM32F030C6	1	32 KB Flash, 4 KB SRAM, 48 MHz, LQFP-48
MG996R Servo	5	11 kg·cm @ 6 V, 4.8–7.2 V, 0.17 s/60°
MG90 Servo	1	2.2 kg·cm @ 4.8 V, 4.8–6 V, 0.1 s/60°
Power Adapter	1	5 V, 3 A output
PCA9685 Driver	1	16-ch PWM, 12-bit, I ² C
ESP32-CAM Module	1	1600×1200, Wi-Fi/BT, OV2640
DC Barrel Jack	1	5.5/2.1 mm, 5–12 V
Jumper Cables	Assorted	M-M, M-F, F-F, 10 cm
Mini Breadboard	1	170 tie-points, single rail

Table 2: Bill of Materials for Electronics Subsystem.

8 Controls and Software

8.1 Embedded Software (STM32F030R6)

The embedded software was developed on the STM32F030R6 microcontroller (ARM Cortex-M0 core) to handle servo actuation and communication with the host PC. The PCA9685 PWM driver was configured over I2C for precise servo control, while UART provided a serial interface for receiving object detection data from the host system.

Key functionalities implemented include:

- System clock configuration at 48 MHz for real-time control.
- I2C initialization for communication with the PCA9685 driver.
- Functions for setting individual, multiple, or all servo angles.
- UART initialization and communication routines for receiving YOLO object detection data.
- Parsing of bounding box coordinates into a format usable for robotic arm control.

Reference: `controls-software/Stm32f030rct6/` (GitHub)

[language=C, caption=STM32 Embedded Code Snippet for Servo Control and UART Communi-

```

cation, basicstyle=]
void PCA9685_Init(void)
PCA9685_WriteReg(PCA9685_MODE1, MODE1_RESTART);
LL_mDelay(10);
uint8_t prescale = (uint8_t)((OSC_FREQ/(4096.0 * SERVO_FREQ)) + 0.5) - 1;
uint8_t oldmode = PCA9685_ReadReg(PCA9685_MODE1);
uint8_t newmode = (oldmode | MODE1_RESTART) & ~MODE1_SLEEP;
PCA9685_WriteReg(PCA9685_MODE1, newmode);
PCA9685_WriteReg(PCA9685_PRESCALE, prescale);
PCA9685_WriteReg(PCA9685_MODE1, oldmode);
LL_mDelay(5);
PCA9685_WriteReg(PCA9685_MODE1, oldmode | MODE1_RESTART | MODE1_AI);
void PCA9685_SetServoAngle(uint8_t servo, uint16_t angle_deg)
uint16_t pulse = 102 + ((angle_deg * 410) / 180);
PCA9685_SetPWM(servo, 0, pulse);

```

[language=C, caption=STM32 UART Parsing for YOLO Data, basicstyle=]

```

void ParseCoordinates(const char* data, int* center_x, int* center_y, int* width, int* height)
*center_x = *center_y = *width = *height = 0; char temp_data[100];
strncpy(temp_data, data, sizeof(temp_data) - 1);
temp_data[sizeof(temp_data) - 1] = '\0';

```

```

char* token = strtok(temp_data, ",");
if (token) *center_x = atoi(token);
token = strtok(NULL, ",");
if (token) *center_y = atoi(token);
token = strtok(NULL, ",");
if (token) *width = atoi(token);
token = strtok(NULL, ",");
if (token) *height = atoi(token);

```

8.2 Machine Vision

The host PC implements object detection and vision-based control using the YOLOv8 deep learning model. The detected object coordinates (center position and bounding box dimensions) are sent to the STM32 over UART for real-time pick-and-place control.

Key functionalities implemented include:

- Live camera feed acquisition and preprocessing.
- YOLO inference for object detection and localization.
- Real-time visualization of bounding boxes and object centers.
- Serial communication of object coordinates to STM32.

Reference: [controls-software/Python Scripts/ \(GitHub\)](#)
[language=Python, caption=YOLO Object Detection with STM32 Communication, basicstyle=]

```

model = YOLO('yolov8n.pt')
cap = cv2.VideoCapture(1)
cap.set(3, 1280)
cap.set(4, 720)
while True:
    success, img = cap.read()
    if not success: continue
    results = model(img, verbose=False)
    for box, cls in zip(results[0].boxes.xyxy.cpu().numpy(),
    results[0].boxes.cls.cpu().numpy()):
        x1, y1, x2, y2 = box.astype(int)
        center_x, center_y = (x1 + x2)//2, (y1 + y2)//2
        width, height = x2 - x1, y2 - y1
        cv2.rectangle(img, (x1, y1), (x2, y2), (0,255,0), 2)
        cv2.circle(img, (center_x, center_y), 5, (0,0,255), -1)
        class_name = model.names[int(cls)]
        cv2.putText(img, class_name, (x1, y1 - 10),
        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
        message = f"center_x, center_y, width, height"
        ser.write(message.encode('utf-8'))
    ser.flush()

```

9 Fabrication and Assembly

The manipulator arm was realized through 3D printing of all required links and joints. STL files were first exported and prepared for slicing, after which the parts were printed on an Ender 3 using PLA filament. Once printed, the components were inspected to ensure dimensional accuracy and assembled into the complete manipulator arm without major fitting issues. Parts were assembled without much difficulty.

The manipulator base platform was fabricated from ABS board to provide a stable mounting surface for the arm. A 24 in × 24 in square grid was designed and then pasted onto the ABS board. This ensured accurate placement of the arm and alignment with the workspace. The base platform was successfully made from ABS board and 24×24 inch SAV grid sheets.

The electronic subsystem was developed on a modular breadboard layout to simplify prototyping and reduce soldering. The STM32F030C6 microcontroller interfaced with the PCA9685 driver over I²C, generating PWM signals for five MG996R servos and one MG90 connected on channels 0–5. All servos shared a regulated 5 V supply distributed across the breadboard rails from a barrel-jack adapter.

UART lines on the STM32 were connected to the host PC for command reception from Python scripts, while the ESP32-CAM was connected via USB/serial to provide real-time image streaming for YOLO-based object detection. The integration process involved validating PWM outputs, testing servo response, and checking communication links between the STM32, ESP32-CAM, and PC. The system performed reliably after assembly, with synchronized mechanical actuation and visual feedback confirmed in the gallery records. The circuit was assembled on a breadboard, tested with the PCA9685 and servos, and verified to interface correctly with both the STM32 and ESP32-CAM modules. All subsystems operated as intended.

10 Results and Analysis

The manipulator was first subjected to motion tests to verify smooth operation across all joints. Individual servos were actuated and then driven in coordinated sequences to evaluate overall range of motion and responsiveness. These trials confirmed that the PCA9685 driver, controlled through the STM32, produced stable PWM signals that resulted in consistent actuation. Videos of these sequences, including pick-and-place demonstrations, were recorded and archived in the project gallery. Payload capacity was then evaluated by gradually increasing the weight lifted at the gripper. The arm handled lightweight objects reliably, though higher loads highlighted servo torque limitations that constrained the maximum payload which was less than 500 grams.

Accuracy and repeatability were assessed by commanding the manipulator to reach predefined target points on the base platform grid. Measurements showed that the end-effector could consistently reach within a few millimeters of the desired location, with repeat trials producing similar results. However, small deviations were observed due to servo backlash and tolerances in the 3D-printed joints. Overall, the system demonstrated stable motion, adequate payload capacity for its intended use, and reasonable accuracy for an educational and demonstrative robotic manipulator.

11 Conclusions and Recommendations

The development of the robotic manipulator successfully demonstrated the integration of embedded control, machine vision, and low-cost fabrication methods. The system achieved smooth joint actuation through the STM32 and PCA9685 driver, with vision-based control enabled by YOLO detection on the host PC. Fabrication using 3D-printed parts and an ABS board platform provided a cost-effective and modular assembly, while circuit prototyping on a breadboard allowed rapid iteration. Motion tests confirmed reliable operation, payload trials showed adequate lifting for lightweight objects, and accuracy tests indicated reasonable repeatability for educational and demonstrative use. Overall, the project achieved its design objectives within the given constraints.

For future improvements, several recommendations are proposed. Mechanical precision can be enhanced by refining 3D-printed joints or introducing metal reinforcements to reduce backlash. Replacing the breadboard layout with a custom PCB would improve reliability and wiring robustness. Higher torque servos or closed-loop actuators could extend payload capacity, while implementing kinematic calibration would improve positioning accuracy. Finally, integrating onboard vision processing directly on an ESP32 or Jetson Nano could reduce latency and enable more autonomous operation. These enhancements would allow the system to scale from a prototype demonstrator to a more capable robotic platform.

A Project repository

<https://github.com/amprebonne/Beta-Arm>

B 3D-MCAD Assembly

<https://autode.sk/4nquLwD>

C Gallery

<https://github@@.com/amprebonne/Beta-Arm/tree/main/gallery>